Lab 6: Binary Search Trees in comparison with Ordered Array and Linked List

Due: 11:59pm on April 27, 2025 (Sunday)

We have learned a few advantages and disadvantages of ordered array, linked list, and binary search tree.

- Using the binary search algorithm, we can search for an item in ordered array quickly. For this reason, ordered arrays are useful when there are a lot of searches but NOT when there are a lot of deletions or insertions because of many shifting operations.
- When it comes to linked list, insertions and deletions are also slow. The main reason why they are slow is that searching for an item in linked list can be slow because it is a sequential access data structure. But there is no need to shift here.
- Binary search tree can be very useful when you need to insert and delete many items AND need to search often.

In this lab, let's try to verify these advantages and disadvantages. To be able to see the difference, what we need to do first is to build a new binary search tree from an ordered array.

Problem

Given an ordered array (ascending order), you know you would have a very undesirable binary search tree if you build it by simply inserting one by one from the first to the last item in the array.

But you would think there should be a better way to build a BST even with this given constraint, if you are aware of the order in advance.

Download the Lab 6 files (`BSTInterface.java, BST.java, BSTComparison.java` and `Stopwatch.java`) from Canvas and implement a recursive `createMinBST` method that builds a binary search tree ***with minimal height*** from an ordered array.

Hint: Think carefully about binary search (not binary search tree!) and what value in the array should be the root of the binary search tree!

As you can see from the following code, you've already written your BST implementation that has insert, delete and find methods.

```
public static BST createMinBST(int[] array) {
    BST theBST = new BST();
    createMinBST(theBST, array, 0, array.length-1);
    return theBST;
}
private static void createMinBST(BST theBST, int[] array, int start, int end)
{
    // your code goes here…
}
```

Once you are done,

Run the `BSTComparison.java` file!

1. What are the computing times for search? Which one is the fastest and which one is the slowest?

BST: 0.0
Ordered Array: 0.0
LinkedList: 67.0 ms

2. What are the computing times for deletion? Which one is the fastest and which one is the slowest?

BST: 0
Ordered Array: 5
LinkedList: 28

3. What are the computing times for insertion? Which one is the fastest and which one is the slowest?

BST: 0
Ordered Array: 4
LinkedList: 0

4. Fill out the cells in the table below for the worst-case running time complexity of each operation of different data structures.

	Searching	**Deletion**	**Insertion**
Ordered Array			
LinkedList			
BST (Balanced)			
BST (Unbalanced)			

In case of searching in ordered array, you can assume that it uses the binary search algorithm. You do not need to submit your answers above, but do make sure you understand the difference in performance.

Submit your `BSTComparison.java` file using Autolab (https://autolab.andrew.cmu.edu).
Do not zip the file, and do not use package statements.

	Searching	**Deletion**	**Insertion**
Ordered Array	O(log n)	O(n)	O(n)
LinkedList	O(n)	O(n)	O(n)
BST (Balanced)	O(log n)	O(log n)	O(log n)
BST (Unbalanced)	O(n)	O(n)	O(n)